

Student Meal Planner

Frontend Design Decisions Research

A Technical Research Report for the Grubs4Scrubs Project

Date: April 2026

Author: Yao Cheng Zhou

Introduction

When I started building the Grubs4Scrubs frontend, the first few pages used a mix of Tailwind CSS utility classes and custom CSS. The Dashboard page was entirely Tailwind. The Home page and Recipes page used custom CSS with descriptive class names. It was inconsistent, and as more pages got added (Meal Planner, Recipe View, Shopping List), the inconsistency became a real problem. Styles didn't match, naming conventions conflicted, and debugging CSS meant checking two completely different systems.

I decided to standardize everything on custom CSS with a consistent naming convention and component structure. This report explains the reasoning behind that decision, what alternatives I considered, and how the final approach works in practice. It's relevant to the Designing phase of the portfolio because it covers visual design consistency, code organization, and the reasoning behind architectural choices on the frontend.

Main Research Question

What is the most maintainable CSS architecture and React component structure for the Grubs4Scrubs frontend?

Sub-Questions, Methods, and Approach

I split this into four sub-questions. The methods were a mix of reading existing documentation, analyzing how other projects handle this, and honestly just trying things in my own codebase to see what worked.

Tailwind vs. Custom CSS: Which Fits Better?

Sub-question: *Should Grubs4Scrubs use Tailwind CSS utility classes or custom CSS with descriptive class names?*

Method: Available product analysis, comparing both approaches already present in the codebase.

I didn't need to go far for this one. I already had both approaches in the same project. The Dashboard page used Tailwind (`bg-white/[0.04] border border-white/[0.06] p-6 rounded-card`), while the Recipes page used custom classes (`.recipe-card`, `.recipe-card-image`, `.recipe-tags`). I compared readability, ease of modification, and how quickly I could understand what each approach was doing when I came back to the code after a week.

What Naming Convention Works for CSS Classes?

Sub-question: *What CSS class naming convention keeps styles organized as the number of pages and components grows?*

Method: Library research on CSS methodologies (BEM, SMACSS) and available product analysis of existing pages.

I looked at established CSS naming methodologies, particularly BEM (Block Element Modifier). Then I compared that to what I was already naturally doing in the Home and Recipes pages. The pattern I'd been using without thinking about it turned out to be close to BEM, just without the double underscores.

How Should React Components Be Structured?

Sub-question: *What's the best way to organize React components within page files for readability and reusability?*

Method: Library research using React documentation and community research on common patterns.

I read the official React docs on component composition, then looked at how other student and small-team projects organize their components. The question was mainly: should each component be its own file, or can multiple components live in the same file? And how should they relate to each other?

How Do You Keep a Dark Theme Consistent Across Pages?

Sub-question: *What's the best approach to maintaining consistent colors, spacing, and visual style across all pages?*

Method: Best good and bad practices analysis, looking at design systems and color palettes.

I looked at how design systems handle color consistency. The problem was concrete: my Home page used `#1a1a2e` for card backgrounds, the Dashboard used Tailwind's `bg-white/[0.04]`, and the Recipes page used a slightly different shade. I needed a way to keep every page looking like it belonged to the same app.

Results: Sources Reviewed

React Documentation: Thinking in React

The official React docs recommend breaking UI into a component hierarchy. Each component should do one thing. If it gets too complex, break it into smaller components. The docs specifically encourage keeping components in the same file when they're tightly related and only used in one place. This matched my instinct to keep `DashboardHeader`, `DashboardMetrics`, and `DashboardMealPlan` all in `DashboardPage.jsx`.

CSS-Tricks: BEM 101

This article explains BEM naming (Block__Element--Modifier) and why it helps with CSS organization. The core idea is that class names should describe what the element is, not how it looks. `.recipe-card-image` tells you more than `.large-rounded-grey-box`. BEM prevents naming collisions because every class name is scoped to its block. I adapted this slightly, using hyphens only (no double underscores) since that matched my existing style.

MDN Web Docs: CSS Custom Properties

MDN's documentation on CSS custom properties (variables) showed how to centralize colors and spacing values. Instead of hardcoding `#1a1a2e` in fifty places, you define it once as a variable and reference it everywhere. This makes theme changes trivial and ensures consistency.

Josh Comeau: CSS Architecture for Modern Web Apps

Comeau's writing on CSS architecture argues against Tailwind for projects where you want to understand the styling at a glance. His point is that `className="bg-white/[0.04] border border-white/[0.06] p-6 rounded-card"` requires you to mentally parse twelve utility classes to understand what a div looks like, while `className="Dashboard-stat-card"` is immediately clear. He's not anti-Tailwind in general, but he argues custom CSS scales better when you have a consistent naming convention.

Smashing Magazine: Component-Driven CSS

This source covers how to organize CSS files alongside their components. The recommendation: one CSS file per page or major component, imported directly in the JSX file. This keeps styles co-located with the components they style, making it easy to find and modify things. It also prevents the "one giant CSS file" problem where everything lives in a single stylesheet.

Conclusion: Sub-Questions

Tailwind vs. Custom CSS, Answered

Custom CSS with descriptive class names is the better fit for Grubs4Scrubs. Tailwind works well for rapid prototyping, but the utility classes make the JSX hard to read and the styles hard to modify selectively. With custom CSS, the markup stays clean (`className="Dashboard-stat-card"`) and the styles live in a dedicated file where they're easy to find. After converting the Dashboard and Meal Planner pages from Tailwind to custom CSS, the JSX became noticeably more readable and the styling easier to adjust.

CSS Naming Convention, Answered

The convention that works best is `ComponentName-element-subelement`. For example: `Dashboard-stat-card`, `Dashboard-stat-label`, `RecipeView-hero-title`. This is a simplified version of BEM that uses only hyphens. The component name prefix prevents class name collisions between pages, and the hierarchy makes it obvious what each class styles. It matched the pattern I was already using on the Home page, which made the transition natural.

React Component Structure, Answered

Each page is one file with multiple function declarations. The default export is the page wrapper (which includes the Navbar and Footer), and internal functions handle sections of the page. For example, `DashboardPage.jsx` exports `Dashboard` and contains `DashboardHeader`, `DashboardMetrics`, `DashboardMealPlan`, `DashboardFeatured`, and `DashboardShoppingList` as separate functions. Components use regular function declarations (not arrow functions) and wrap their returns in fragments (`<>` `</>`). Data flows down through props.

Dark Theme Consistency, Answered

A fixed color palette used across all CSS files keeps things consistent. The core colors are: `#111125` (page background), `#1a1a2e` (card backgrounds), `#0f0f1b` (darker sections), `#38bdf8` (primary blue), `#ffb95f` (secondary orange), `#d4c4ff` (tertiary purple), `#8a8f98` (muted text), and `#c9cdd4` (body text). Common patterns like glass cards (`rgba(255, 255, 255, 0.04)` background with `rgba(255, 255, 255, 0.06)` borders) and hover effects (`translateY(-4px)` with `border-color: #4f46e5`) are repeated identically across pages.

Conclusion: Main Question

The most maintainable frontend approach for Grubs4Scrubs is custom CSS with `ComponentName-element-subelement` naming, one CSS file per page, and React components organized as multiple named functions within each page file. This replaced the inconsistent mix of Tailwind and custom CSS that existed before.

The practical impact was significant. Converting the Dashboard from Tailwind to custom CSS reduced the JSX from dense, hard-to-read utility strings to clean semantic class names. Adding new pages (Meal Planner, Recipe View) became predictable because the pattern was established: create the page component, break it into named functions, create a matching CSS file, use the same color palette and naming convention. The codebase went from feeling like three different people wrote it to feeling cohesive. For the portfolio, this decision is easy to document and justify because the before/after difference is visible in the code.

Summary

Grubs4Scrubs standardized its frontend on custom CSS (no Tailwind) with a `ComponentName-element-subelement` naming convention, one CSS file per page, and React components structured as multiple named functions per file. The dark theme uses a fixed color palette applied consistently across all pages. This approach prioritizes readability, maintainability, and visual consistency over the rapid development speed that Tailwind offers.

References

All references are formatted in APA 7th edition style.

React. (2024). *Thinking in React*. React Documentation. <https://react.dev/learn/thinking-in-react>

Rendle, R. (2023). BEM 101. *CSS-Tricks*. <https://css-tricks.com/bem-101/>

MDN Web Docs. (2024). *Using CSS custom properties (variables)*. Mozilla Developer Network. https://developer.mozilla.org/en-US/docs/Web/CSS/Using_CSS_custom_properties

Comeau, J. (2024). *CSS architecture for modern web apps*. Josh W Comeau. <https://www.joshwcomeau.com/css/>

Verou, L. (2023). Component-driven CSS patterns. *Smashing Magazine*. <https://www.smashingmagazine.com/>